

Phase 1 Report

Kubernetes Workload Analyzer — local cluster, Kubernetes fundamentals and verified monitoring

Phase status	DONE (2026-09-24)
Report date	2026-09-25
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	63454cc (PR #1 and PR #2 merged)
Cluster	kind workload-analyzer , Kubernetes v1.37.0, single node
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 1 was for
2. Outcome against the done-criteria
3. Machine and tooling
4. The kind cluster
5. Lab 01 — Deployment, ReplicaSet, Pod
6. Lab 02 — Service and probes
7. Lab 03 — Resources, QoS, OOM, throttling, scheduling
8. Lab 04 — ConfigMap, Secret, RBAC
9. metrics-server
10. Lab 05 — HPA and the request coupling
11. The monitoring stack
12. Lab 06 — Verifying every metric
13. Security verification
14. What Phase 1 changed in the analyzer design
15. Mistakes and corrections
16. Git and GitHub workflow
17. Current state, operating and rollback
18. Open items and next phase
19. Appendix A — Command cheat sheet
20. Appendix B — Glossary

1. What Phase 1 was for

The project builds an analyzer that turns Kubernetes and Prometheus data into evidence-based findings (for example "this workload requests far more CPU than it uses"). Phase 1 did not write analyzer code. It had two jobs:

- 1. **Learn the Kubernetes behaviour the analyzer will reason about**, by running small, deliberate experiments and writing down what actually happened, not what documentation suggests.
- 2. **Build and verify the data layer**: a local cluster plus a trimmed Prometheus stack, and prove that every metric the planned rules (R001-R005) depend on really exists and behaves as expected.

Prometheus was installed at the end of Phase 1 on purpose (decision taken 2026-09-24): rightsizing rules need days of history, so the history clock should start as early as possible, while later phases are being built.

Decisions in force during Phase 1

ID	Decision	Status
D-001	Backend in Go (toolchain 1.27)	ACCEPTED
D-002	Dashboard: React + Vite + TypeScript + uPlot, served as static files by the backend; Grafana for raw time series	ACCEPTED
D-003	Trimmed kube-prometheus-stack with persistent storage	ACCEPTED
D-004	No application database in the MVP	PROPOSED
D-005	kind, single node (k3d fallback if RAM gets tight)	ACCEPTED
D-006	Portfolio/demo positioning: unified, explainable findings; KRR/OpenCost as baselines	ACCEPTED
D-007	Backend API access: read-only ClusterRole on structure + Events; no pods, secrets, configmaps	ACCEPTED

2. Outcome against the done-criteria

Done-criterion (PHASES.md)	Result
Tools installed; one-node cluster created	Done. kubectl v1.37.0, kind v0.33.0, Helm v4.3.0 (checksums verified); cluster Ready after 16 s
Every concept has a lab-notes entry	Done for Pod, Deployment, ReplicaSet, Service, Namespace, ConfigMap, Secret, requests/limits, probes, labels/selectors, Events, RBAC, HPA, scheduling, pod-level failure/recovery. Cluster-level recovery (Docker restart, reboot) was moved to Phase 7.
Prometheus footprint MEASURED	Done. Monitoring pods ~580 MiB; ~15.5k active series; ~600 samples/s
All measurement-model metrics return data	Done, under real conditions (OOM, crash loop, Pending, throttling, HPA)
kube-state-metrics cannot list secrets	Done. <code>auth can-i → no</code> ; zero <code>kube_secret_*</code> series

3. Machine and tooling

3.1 Machine MEASURED (2026-09-23)

Item	Value
OS	Ubuntu 26.04.1 LTS, kernel 7.0.0-31-generic, x86_64
CPU	Intel i5-1235U: 10 physical cores (2 performance cores with hyper-threading + 8 efficiency cores), 12 logical CPUs
RAM	14.79 GiB total; roughly 6.5–10 GiB available depending on open desktop apps; 4 GiB swap file
Disk	ext4, 140 GB, 75 GB free after cleanup
Docker	29.1.3, cgroup v2, containerd image store (image data lives in /var/lib/containerd)
inotify	max_user_instances=128, max_user_watches=65536 — fine for one node; a known problem for multi-node kind
Sleep	68 suspends in 14 days; lid close suspends. Suspends create gaps in Prometheus history

Main constraint: memory. The cluster budget was set at about 4.5 GiB so the desktop keeps enough headroom.

3.2 Tool installation

All three tools were installed into `~/.local/bin` (no sudo, nothing system-wide), and each download was checked against its published SHA-256 checksum before installation. `~/.bashrc` line 126 adds the folder to `PATH`.

```
mkdir -p ~/.local/bin
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
curl -fsSL https://dl.k8s.io/release/v1.37.0/bin/linux/amd64/kubectl
echo "$(curl -fsSL https://dl.k8s.io/release/v1.37.0/bin/linux/amd64/kubectl.sha256) kubectl" | sha256sum -c
curl -fsSL https://kind.sigs.k8s.io/dl/v0.33.0/kind-linux-amd64
curl -fsSL https://kind.sigs.k8s.io/dl/v0.33.0/kind-linux-amd64.sha256sum | awk '{print $1" kind"}' | sha256sum -c
curl -fsSL https://get.helm.sh/helm-v4.3.0-linux-amd64.tar.gz
curl -fsSL https://get.helm.sh/helm-v4.3.0-linux-amd64.tar.gz.sha256sum | sha256sum -c
tar -xzf helm-v4.3.0-linux-amd64.tar.gz linux-amd64/helm
install -m 0755 kubectl kind linux-amd64/helm ~/.local/bin/
```

Tool	Version	Purpose
kubectl	v1.37.0 (bundles Kustomize v5.8.1)	Client for the Kubernetes API; every lab uses it
kind	v0.33.0	Runs a whole Kubernetes cluster inside one Docker container
helm	v4.3.0	Installs charts; used for kube-prometheus-stack (Helm 4 compatibility confirmed)

Removal: `rm ~/.local/bin/{kubectl,kind,helm}`.

4. The kind cluster

What kind does. kind ("Kubernetes in Docker") starts a Docker container that behaves like a machine. Inside it run containerd, the kubelet and the control-plane components (API server, etcd, scheduler, controller manager), set up by kubeadm exactly as on a real node. Pods run as containers *inside* that container.

```
kind create cluster --name workload-analyzer --wait 180s
kubectl cluster-info --context kind-workload-analyzer
kubectl get nodes -o wide
kubectl get pods -A
```

kind prefixes the cluster name with `kind-` to form the kubectl context: cluster `workload-analyzer` → context `kind-workload-analyzer`, node container `workload-analyzer-control-plane`.

4.1 What was created

Namespace	Pods	Role
kube-system	kube-apiserver, etcd, kube-scheduler, kube-controller-manager, kube-proxy, kindnet (CNI), coredns x2	Control plane, networking, DNS
local-path-storage	local-path-provisioner	Creates PersistentVolumes as folders inside the node (StorageClass <code>standard</code> , default)

4.2 Measured footprint **MEASURED**

Item	Value	Meaning
Idle cluster memory	~550 MiB (547 / 548 / 551)	Below the 0.6–1.0 GiB estimate made in Phase 0
Idle cluster CPU	21–25% of one core	Control-plane background work
Create time	2 min 49 s; Ready after 16 s	Almost all of it was the first pull of the 1.34 GB node image
Node allocatable	12 CPU, 14.79 GiB	The node believes it owns the whole laptop. Workload memory limits are what protect the desktop.
Node container restart policy	<code>on-failure:1</code>	Whether the cluster returns after a reboot is UNVERIFIED (Phase 7 test)
<code>~/.kube/config</code>	mode 600	Admin credentials, readable only by the owner; outside the repo and in <code>.gitignore</code>

Incident: a second cluster. Shortly after creation a second cluster called `k8sa` appeared (most likely the earlier preview command was run in a terminal). Creating a kind cluster switches kubectl's current context to it, so commands silently went to the new, empty cluster and the `learn` namespace looked "missing". Nothing was lost. `k8sa` was deleted with owner approval and the context switched back. **Lesson:** every command after that used `--context kind-workload-analyzer` explicitly.

5. Lab 01 — Deployment, ReplicaSet, Pod, labels, self-healing, scaling

Concept. A **Pod** is one or more containers scheduled together. A **ReplicaSet** keeps a fixed number of identical Pods alive. A **Deployment** manages ReplicaSets so the pod template can be changed safely (rolling updates). **Labels** are key/value tags; **selectors** are how controllers and Services find the Pods they own.

Manifest `learn/01-deployment.yaml` (image `registry.k8s.io/e2e-test-images/agnhost:2.53`, the Kubernetes project's own test image: Alpine-based, has `sh/curl`, serves HTTP, no Docker Hub rate limits):

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: web, namespace: learn, labels: {app: web}}
spec:
  replicas: 2
  selector: {matchLabels: {app: web}}      # which pods this Deployment owns
  template:
    metadata: {labels: {app: web}}        # must match the selector
    spec:
      containers:
      - name: web
        image: registry.k8s.io/e2e-test-images/agnhost:2.53
        args: ["netexec", "--http-port=8080"]
        ports: [{containerPort: 8080}]
        resources:
          requests: {cpu: 50m, memory: 32Mi}
          limits: {memory: 64Mi}
```

Experiment	Command	Observed
Apply	<code>kubectl -n learn get deploy,rs,pods --show-labels</code>	Deployment <code>web</code> → ReplicaSet <code>web-5b8dbf6fc9</code> → 2 Pods; <code>ownerReferences</code> confirm the chain
Labels	same	All carry <code>app=web</code> ; RS and Pods also carry <code>pod-template-hash=5b8dbf6fc9</code> , derived from the template
Delete a pod	<code>kubectl -n learn delete pod <name></code>	Replacement Running within ~3 s, new name and new IP
Event order	<code>kubectl -n learn get events --sort-by=.lastTimestamp</code>	SuccessfulCreate of the replacement appears <i>before</i> Killing of the old pod
Scale 2 → 3	<code>kubectl -n learn scale deploy web -replicas=3</code>	Event <code>ScalingReplicaSet ... from 2 to 3</code>
Image pull	events	First pull 22 s (54 MB); later pods: "already present on machine"

Why it matters for the analyzer. Pods are disposable: names and IPs change on every replacement, and a terminating pod stops counting toward replicas before it is gone, so pod counts can briefly exceed the replica count. Findings must be keyed by **workload** (Deployment), never by pod name.

6. Lab 02 — Service, EndpointSlice, rolling update, readiness vs liveness

Concept. A **Service** gives a stable virtual IP and DNS name to a changing set of Pods, chosen by label selector; the actual Pod IPs are listed in **EndpointSlices**. A **readiness probe** decides whether a Pod receives traffic. A **liveness probe** decides whether the kubelet restarts the container.

Manifest `learn/02-service-probes.yaml` updates the same Deployment. The probes check marker files so that a single pod can be broken on purpose (real applications would use `httpGet` on a health endpoint):

```
command: ["/bin/sh", "-c"]
args: ["touch /tmp/ready /tmp/alive && exec /agnhost netexec --http-port=8080"]
readinessProbe:
  exec: {command: ["cat", "/tmp/ready"]}
  periodSeconds: 5
  failureThreshold: 1
livenessProbe:
  exec: {command: ["cat", "/tmp/alive"]}
  periodSeconds: 5
  failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata: {name: web, namespace: learn}
spec:
  selector: {app: web}
  ports: [{port: 80, targetPort: 8080}]
```

Step	Observed
A. Rolling update	New ReplicaSet <code>web-77765dc8b5</code> 3/3; old one scaled to 0 but kept (rollback possible); rollout history shows revisions 1 and 2
B. Service	ClusterIP <code>10.96.113.85:80</code> ; EndpointSlice lists 3 pod IPs on port 8080
C. Load balancing	After the rollout: 9 calls spread 4/3/2 across pods, 0 failures. During the rollout one call returned empty and stalled ~2 min (likely a terminating pod; UNVERIFIED) — later calls used <code>--max-time 3</code>
D. Readiness broken (<code>rm /tmp/ready</code>)	Pod <code>0/1</code> , <code>ready=false</code> in EndpointSlice, all traffic to the other two (5/4), restart count stays 0
E. Liveness broken (<code>rm /tmp/alive</code>)	Restart after 13 s (3 × 5 s). Events: Unhealthy: Liveness probe failed: cat: can't open '/tmp/alive', then Killing: Container web failed liveness probe, will be restarted
Termination record	<code>lastState.terminated: reason: "Completed", exitCode: 0</code>

Key finding: a liveness kill looks like a clean exit. The kubelet sent SIGTERM and the app shut down cleanly, so the recorded reason is `Completed` with exit code 0. kube-state-metrics reports exactly this reason. Consequences:

- R003 (frequent restarts) must count **restart increases**; it cannot rely on termination reasons.
- The **cause** of such a restart exists only in Events — concrete support for D-007.
- Readiness failures cause no restarts at all; they show up as unavailable replicas.

7. Lab 03 — Requests/limits, QoS, OOM, throttling, scheduling

Concept. A **request** is what the scheduler reserves for a container; a **limit** is the ceiling the kernel enforces. Memory above the limit triggers the kernel OOM killer. CPU above the limit is **throttled** (paused for the rest of each 100 ms period), never killed. The **QoS class** is derived from how requests and limits are set and decides eviction order under node memory pressure.

Manifest `learn/03-resources.yaml`: seven standalone pods, all using `agnhost`'s built-in `stress` (`--cpus N` whole CPUs, `--mem-total`) or `pause`, so no extra image was needed.

```
# OOM: allocates 200Mi against a 64Mi limit
  args: ["stress", "--mem-total=200Mi", "--mem-alloc-size=1Mi", "--mem-alloc-sleep=10ms"]
  resources: {requests: {cpu: 50m, memory: 64Mi}, limits: {memory: 64Mi}}
# Throttling: wants 1 full CPU, limit allows 200m
  args: ["stress", "--cpus=1"]
  resources: {requests: {cpu: 100m, memory: 32Mi}, limits: {cpu: 200m, memory: 64Mi}}
# Pending 1: requests more CPU than the node has
  resources: {requests: {cpu: "64"}}
# Pending 2: requires a node label that does not exist
  nodeSelector: {disktype: ssd}
```

Experiment	Observed	Command
QoS classes	BestEffort (no resources); Burstable (request < limit, or request only); Guaranteed (request = limit)	<code>kubectl get pods -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass</code>
OOM	reason: <code>OOMKilled</code> , <code>exitCode: 137</code> , ~1 s after each start; 3 restarts in 49 s, then <code>BackOff</code>	<code>kubectl get pod oom -o jsonpath='{.status.containerStatuses[0].lastState}'</code>
CPU throttling	<code>cpu.max = 20000 100000 (= 200m)</code> . <code>nr_throttled == nr_periods</code> (191/191, then 246/246): every period throttled. Usage over 5 s: 1.087 s CPU ≈ 217m	<code>kubectl exec throttled -- cat /sys/fs/cgroup/cpu.stat</code>
Pending (resources)	<code>FailedScheduling: 0/1 nodes are available: 1 Insufficient cpu</code> . <code>preemption: ... not helpful</code>	<code>kubectl get events --field-selector reason=FailedScheduling</code>
Pending (selector)	didn't match Pod's node affinity/selector; scheduled immediately after <code>kubectl label node ... disktype=ssd</code>	<code>kubectl label node <n> disktype=ssd</code>
Label removed	Pod kept running : selectors are only checked at scheduling time	<code>kubectl label node <n> disktype-</code>
Taint NoSchedule	New pod Pending (<code>untolerated taint(s)</code>); existing pods unaffected; scheduled after <code>untaint</code>	<code>kubectl taint node <n> lab=only:NoSchedule[-]</code>
Allocated	Node requests 1350m CPU (11%) / 546Mi; Pending pods are not counted ; CPU limits 250m	<code>kubectl describe node <n></code>

Why it matters for the analyzer.

- A real OOM is reliably recorded as `OOMKilled / 137` — unlike a liveness kill (`Completed / 0`). R004's main signal works.
- The throttling ratio from the kernel's cgroup counters is exactly what the Prometheus metrics expose; 100% throttling means the workload needs more CPU than allowed — R001 must never recommend shrinking it.
- Scheduling reasons exist only in Events; "requested" totals must exclude Pending pods.

8. Lab 04 — ConfigMap, Secret, RBAC

Concept. A **ConfigMap** holds non-secret configuration; a **Secret** holds sensitive values. Both can be injected as environment variables or mounted as files. **RBAC** grants verbs (get, list, watch, create, delete ...) on resources to identities such as ServiceAccounts, through Roles/ClusterRoles and their bindings.

The Secret was created by command so no value ever reached git:

```
kubectl -n learn create secret generic web-secret --from-literal=api-token="$(openssl rand -hex 8)"
```

Manifest `learn/04-config-secret-rbac.yaml` contains a ConfigMap, a pod with three env variables (a literal value on purpose, a ConfigMap reference, a Secret reference) and a mounted ConfigMap, plus the **real D-007 RBAC**:

```
kind: ClusterRole
metadata: {name: analyzer-readonly}
rules:
- apiGroups: [""]
  resources: [namespaces, nodes, events]
  verbs: [get, list, watch]
- apiGroups: ["apps"]
  resources: [deployments, replicaset, statefulsets, daemonsets]
  verbs: [get, list, watch]
- apiGroups: ["autoscaling"]
  resources: [horizontalpodautoscalers]
  verbs: [get, list, watch]
- apiGroups: ["events.k8s.io"]
  resources: [events]
  verbs: [get, list, watch]
```

Experiment	Observed
A. Secret storage	Opaque, data in base64. Plain <code>base64 -d</code> recovered the 16-character value, identical to <code>API_TOKEN</code> inside the container (value never printed). Base64 is encoding, not encryption.
B. Pod spec exposure	<code>LITERAL_PASSWORD=visible-in-pod-spec</code> in plain text; ConfigMap and Secret values appear only as <code>configMapKeyRef</code> / <code>secretKeyRef</code>
C. ConfigMap update	Mounted file changed to <code>hello-v2</code> after 89 s ; env variable stayed <code>hello-v1</code> (fixed at container start)
D. auth <code>can-i</code> as the ServiceAccount	yes: deployments, events, HPAs, nodes. no: pods, secrets, configmaps, delete, create, pods/log
E. Real token, isolated kubeconfig	<code>auth whoami</code> → <code>system:serviceaccount:learn:analyzer-readonly</code> . Deployments and Events readable; pods, secrets and delete → Forbidden

```
TOKEN=$(kubectl -n learn create token analyzer-readonly --duration=10m)
KUBECONFIG=/dev/null kubectl --server="$S" --token="$TOKEN" --insecure-skip-tls-verify=true auth whoami
KUBECONFIG=/dev/null kubectl --server="$S" --token="$TOKEN" --insecure-skip-tls-verify=true get pods -n learn
# Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:learn:analyzer-readonly" cannot list ...
```

Testing pitfall found. The first token test wrongly showed pods and secrets as readable. `kubectl` still loaded the kubeconfig, sent the admin client certificate along with the token, and the API server authenticated the request as admin. Fix: run with `KUBECONFIG=/dev/null` and assert identity with `kubectl auth whoami` first. The analyzer's RBAC tests must do the same.

Why it matters. Read access to pods exposes literal env values, which often contain secrets. This is why D-007 grants no pods access. The D-007 ClusterRole was verified exactly as designed.

9. metrics-server

What it is. metrics-server scrapes the kubelets every 15 s and serves only the **latest** CPU/memory values through the `metrics.k8s.io` API. `kubectl top` and the HPA read it. It stores no history — which is exactly why D-003 chose Prometheus for the analyzer.

Installed from the pinned upstream manifest via `deploy/metrics-server/kustomization.yaml`:

```
resources:
- https://github.com/kubernetes-sigs/metrics-server/releases/download/v0.9.0/components.yaml
patches:
- target: {kind: Deployment, name: metrics-server, namespace: kube-system}
  patch: |-
    - op: add
      path: /spec/template/spec/containers/0/args/-
      value: --kubelet-insecure-tls    # kind only: kubelet certs are self-signed
```

Fact	Value
Version / image VERIFIED	v0.9.0, <code>registry.k8s.io/metrics-server/metrics-server:v0.9.0</code>
Permissions	cluster-wide get/list/watch on pods and nodes; get nodes/metrics; registers the <code>metrics.k8s.io</code> APIService
Requests vs use MEASURED	requests 100m / 200Mi; uses ~15 Mi
Node memory	669 → 706 MiB (+37 MiB)
--kubelet-insecure-tls	Acceptable only in a local lab; never on a real cluster

First real over-requested workloads. `kubectl top` showed the web pods at 5–6m CPU and 6 Mi memory against requests of 50m / 32Mi ($\approx 8\times$ CPU, $\approx 5\times$ memory), and metrics-server itself at 15 Mi against a 200Mi request ($\approx 13\times$). These are exactly the patterns R001/R002 are meant to find.

10. Lab 05 — HPA and the request/utilization coupling

Concept. The HorizontalPodAutoscaler computes $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentUtilization} / \text{targetUtilization})$, with a 10% tolerance and a 5-minute scale-down stabilization window by default. For type: Utilization targets, utilization is measured **as a percentage of the request**. So changing a request changes utilization — and the replica count. **VERIFIED** (kubernetes.io)

Manifest `learn/05-hpa.yaml`: Deployment `hpa-demo` using `registry.k8s.io/e2e-test-images/resource-consumer:1.15.0` (the image Kubernetes' own HPA tests use), a Service, an HPA (min 1, max 5, target 70% CPU), and a load pod that sends `3 × POST /ConsumeCPU?millicores=100&durationSec=30` every 30 s through the Service — about 300m total, **shared** across replicas like real traffic.

Phase	Request	Measured	HPA result
Start	400m	First ~45 s: no metrics returned / pods might be unready	no action (new pod has no data)
A. Baseline	400m	302m used → 75%	stays at 1 ($75/70 = 1.07$, within tolerance)
B. "Rightsize"	200m	300m → 150%	SuccessfulRescale: New size: 3 = $\text{ceil}(1 \times 150/70) = \text{ceil}(2.14)$
After scale-out	200m	40–62% (80–125m per pod)	stays at 3 (scale-down would need $< \sim 47\%$ sustained for 5 min)

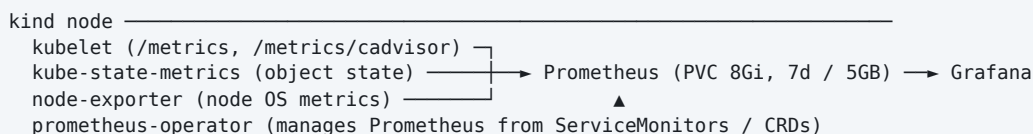
	Before (400m × 1)	After (200m × 3)
Total CPU requested	400m	600m (+50%)
Total memory requested	64Mi	192Mi (3×)
Pods	1	3

Key finding. Cutting the per-pod request on an HPA-managed workload **increased** total allocation. R001 must therefore emit an "HPA-coupled" caveat plus the **HPA-neutral request** ($\text{usage} \div \text{target} = 300\text{m} \div 0.7 \approx 430\text{m}$), never a plain resize. A snapshot showed per-pod usage 190m / 0m / 1m: with few requests, random Service routing leaves some pods idle — use workload-level aggregates for HPA-managed workloads.

Note: the pre-run prediction was "2 replicas". That was an arithmetic error ($150/70 = 2.14 \rightarrow 3$); the HPA followed its formula exactly.

11. The monitoring stack

11.1 Architecture



Component	Provides
cAdvisor (inside the kubelet)	Container usage: CPU seconds, memory working set, CPU throttling counters
kube-state-metrics (KSM)	Object state from the API: requests/limits, restarts, termination/waiting reasons, phases, owners, deployments, HPAs, node conditions
node-exporter	Node-level OS metrics (memory available, OOM-kill counter)
Prometheus operator	Turns ServiceMonitor objects into Prometheus scrape configuration
Grafana	Dashboards for raw time-series investigation (not the analyzer UI)

11.2 Values file — every setting explained

deploy/prometheus/values.yaml for kube-prometheus-stack **91.5.0**:

Setting	Why
alertmanager.enabled: false	No alert routing in the MVP; saves memory
kubeEtcd / kubeScheduler / kubeControllerManager / kubeProxy / kubeApiServer: enabled: false	Control-plane targets are not needed for workload analysis and produce many series
grafana.admin.existingSecret: grafana-admin	Password comes from a Secret created with <code>openssl rand</code> ; never in values or git; stable across <code>helm upgrade</code>
grafana.resources (128Mi request, 384Mi limit)	Bound Grafana's memory
kube-state-metrics.collectors (explicit list without secrets, configmaps)	The defaults include both, and the chart's RBAC grants list/watch on them — which returns Secret values . The RBAC template follows this list, so the permissions disappear too. VERIFIED in the chart's <code>role.yaml</code>
retention: 7d, retentionSize: 5GB	7 days is what HIGH confidence needs; size cap stays under the PVC
Prometheus requests 200m / 512Mi, limit 1Gi	Protects the laptop; Prometheus is the component most likely to grow
storageSpec.volumeClaimTemplate 8Gi	The default is emptyDir: history would be lost on every pod restart

11.3 Installation

```
kubectl create namespace monitoring
kubectl -n monitoring create secret generic grafana-admin \
  --from-literal=admin-user=admin --from-literal=admin-password="$(openssl rand -base64 24)"
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kps prometheus-community/kube-prometheus-stack --version 91.5.0 \
  -n monitoring -f deploy/prometheus/values.yaml --wait --timeout 10m
```

Install took 4 min 32 s. `helm --wait` returned before the Prometheus pod itself was Ready (that pod is created by the operator, not by Helm); it became Ready about 4.5 minutes after starting. The chart printed a note

about a `kps-grafana` secret, but no such secret was created: Grafana's env variables reference `grafana-admin`.
MEASURED

11.4 Measured footprint **MEASURED**

Item	Value
Grafana pod	~400 MiB: grafana 246 + dashboard sidecar 73 + datasource sidecar 72
Prometheus	85 MiB at start → 126 MiB after ~20 min → 144 MiB after ~50 min (+10 MiB config-reloader); limit 1Gi
Operator / KSM / node-exporter	~28 / ~20 / ~11 MiB
Monitoring pods total	~580 MiB
Node container (docker stats)	~770 MiB → ~2.2 GiB; includes reclaimable file cache from image pulls, so it overstates the real cost
Host available memory	7098 → 6274 MiB (–824 MiB; desktop apps also changed)
Active series / ingestion	13.7k → 15.5k series; ~600 samples/s (far below the ~100k-series estimate from Phase 0)
Disk	2.8 MB after ~20 min, 12 MB after ~50 min. Projection: ~80 MB/day, ~0.6 GB for 7 days ESTIMATED

12. Lab 06 — Verifying every metric

Queries were sent through the API server's service proxy, so no port-forward had to stay open:

```
P=/api/v1/namespaces/monitoring/services/kps-kube-prometheus-stack-prometheus:9090/proxy/api/v1
kubectl get --raw "$P/query?query=<url-encoded PromQL>"
```

The first pass found six metrics **absent**. They were not broken: kube-state-metrics and cAdvisor only emit them while the condition exists. The conditions were triggered for ~4 minutes by re-applying the Lab 03 pods and the Lab 05 HPA (without load), then queried again.

Metric	Source	Result
container_cpu_usage_seconds_total, container_memory_working_set_bytes	cAdvisor	present (57 series); throttled pod 172m vs 200m limit
container_cpu_cfs_throttled_periods_total / _cfs_periods_total	cAdvisor	only for CPU-limited containers; ratio = 1.0 (100%). Extra pod-level series with empty container label
container_cpu_cfs_throttled_seconds_total	cAdvisor	absent — dropped by the chart's relabelling (expected)
kube_pod_container_status_restarts_total	KSM	OOM pod: 5
kube_pod_container_status_last_terminated_reason	KSM	oom → OOMKilled; web-...-fx9dw → Completed (the Lab 02 liveness kill)
container_oom_events_total	cAdvisor	0 through 5 real OOMs → not usable
node_vmstat_oom_kill	node-exporter	increases, but node-wide — cannot be attributed to a container
kube_pod_container_status_waiting_reason	KSM	CrashLoopBackOff — only visible with max_over_time(...[6m]); an instant query missed it between restarts
kube_pod_status_unschedulable	KSM	pending-cpu, pending-selector = 1
kube_pod_owner, kube_replicaset_owner, kube_pod_info	KSM	present (18 / 8 / 18)
kube_pod_container_resource_requests / _limits	KSM	present (23 / 7)
kube_deployment_spec_replicas / _status_replicas_available	KSM	present (7 / 7)
kube_node_status_condition / _allocatable	KSM	present (12 / 6)
kube_horizontalpodautoscaler_info / _spec_target_metric	KSM	hpa-demo → Deployment/hpa-demo; cpu Utilization 70

Three rules for writing queries, learned here.

1. **Absent series ≠ missing data.** For state metrics (waiting reason, unschedulable, HPA, throttling) no series means "condition not present". For usage metrics, no series means missing data → R005 insufficient.
2. **Filter container!=""**: cAdvisor adds a pod-level series next to each container series.
3. **Use range queries for short states** (max_over_time): an instant query can fall between restarts.

13. Security verification

Check	Command	Result
KSM can list/watch secrets	<code>kubectl auth can-i list secrets -A --as=system:serviceaccount:monitoring:kps-kube-state-metrics</code>	no (watch: no; configmaps: no; pods: yes, needed)
KSM ClusterRole rules	<code>kubectl get clusterrole kps-kube-state-metrics -o yaml</code>	no secrets/configmaps entries
Secret/ConfigMap metrics	<code>count({__name__=~"kube_(secret configmap)_.*"})</code>	0 series
Grafana anonymous	<code>curl localhost:3300/api/user</code> (temporary port-forward)	401
Grafana wrong password	<code>curl -u admin:wrong ...</code>	401
Grafana admin	<code>curl -u "admin:\$PW" ...</code>	<code>{"login":"admin","isGrafanaAdmin":true}</code>
Grafana image	<code>kubectl exec ... -- sh</code>	no shell (minimal image) — good
Exposure	<code>kubectl -n monitoring get svc</code>	all Services ClusterIP; reached only via port-forward
Analyzer RBAC (Lab 04)	real token, <code>KUBECONFIG=/dev/null</code>	pods, secrets, delete → Forbidden

Components that can still read pod specs: kube-state-metrics (needs pods for pod metrics) and metrics-server. Both are upstream system components and trusted in the lab. The analyzer itself has no `pods` access (D-007). Recorded in `docs/threat-model.md`.

14. What Phase 1 changed in the analyzer design

Evidence	Design consequence
Liveness kill → Completed / 0 (Lab 02)	R003 counts restart increases, not termination reasons; restart <i>causes</i> come from Events
OOM → OOMKilled / 137 (Lab 03, Lab 06)	R004 = KSM <code>last_terminated_reason="OOMKilled"</code> + restart increase
<code>container_oom_events_total</code> stays 0 (Lab 06)	Removed from the measurement model
100% throttling at the limit (Lab 03)	R001's "throttled ratio < 5%" gate is essential
HPA 1 → 3, CPU +50%, memory 3× (Lab 05)	R001 emits "HPA-coupled" + HPA-neutral request ($\text{usage} \div \text{target}$) for Utilization-type HPAs
Uneven per-pod load under an HPA (Lab 05)	Use workload-level aggregates, not single pods
New pod has no metrics for ~45 s (Lab 05)	R005 handles new workloads explicitly
Absent state series (Lab 06)	"No series" semantics differ between state and usage metrics
Scheduling and probe reasons only in Events (Labs 02–03)	Supports D-007 (Events read access)
Pod names/IPs change on replacement (Lab 01)	Findings keyed by workload; stable IDs from cluster + rule + namespace + workload + container
kubeconfig credentials override <code>--token</code> (Lab 04)	RBAC tests run with an isolated kubeconfig and assert <code>auth whoami</code>
Node allocatable = whole laptop (\$4)	Demo workloads always carry memory limits

15. Mistakes and corrections

Recorded so the same traps are not repeated.

What went wrong	Effect	Correction
Second cluster <code>k8sa</code> switched the <code>kubectl</code> context	Commands hit an empty cluster	Deleted <code>k8sa</code> ; <code>--context</code> on every command
Token test with kubeconfig loaded	False "access granted" for pods/secrets	<code>KUBECONFIG=/dev/null + auth whoami</code>
<code>curl</code> without timeout during a rollout	One call stalled ~2 min	<code>--max-time 3</code>
HPA output parsed with <code>awk</code>	"replicas=5" was really MAXPODS (column shift)	Read <code>.status</code> fields with <code>jsonpath</code>
HPA prediction arithmetic	Predicted 2 replicas; actual 3	$\text{ceil}(2.14) = 3$; noted in lab notes
<code>gh pr edit</code> on gh 2.46	Fails on a retired GitHub API	Use <code>gh api -X PATCH repos/.../pulls/N</code>
PR #1 already merged by the owner; merge call was a no-op; its description then edited	3 commits missing from main; PR #1 described content it did not contain	PR #1 description restored; PR #2 opened and merged

16. Git and GitHub workflow

Commit	Content
878dc6e	Phase 0 foundation and accepted decisions (on main)
d82608c	Cluster footprint and labs 01-03
b2d1976	Lab 04 ConfigMap, Secret, read-only RBAC
6ea7ca8	Merge of PR #1 (merged by the owner on GitHub)
0b89a43	metrics-server v0.9.0
2638d88	Lab 05 HPA
b30df45	Trimmed kube-prometheus-stack and verified measurement model; Phase 1 DONE
63454cc	Merge of PR #2 — current main

Conventions: work on a branch (phase-1), stage files by name (never `git add -A`), review `git diff --cached` for secrets before each commit, Conventional-Commit style messages, no AI co-author trailer, merge through a pull request with a merge commit.

```
git switch -c phase-1
git add <files> && git diff --cached --stat
git commit -m "docs(phase-1): ..."
git push -u origin phase-1
gh pr create --draft --base main --head phase-1 --title "..." --body-file body.md
gh api -X PATCH repos/ISHANT57/kubernetes-workload-analyzer/pulls/1 -F "body=@body.md"
gh api -X PUT repos/ISHANT57/kubernetes-workload-analyzer/pulls/2/merge -f merge_method=merge
git switch main && git pull --ff-only
```

17. Current state, operating and rollback

17.1 State at 2026-09-25 00:33 MEASURED

Namespace	Running
kube-system	9 pods (control plane, CoreDNS, kindnet, kube-proxy, metrics-server)
local-path-storage	1 pod
learn	3 web pods + Service web
monitoring	5 pods (Grafana 402 Mi, Prometheus 144 Mi, operator, KSM, node-exporter); PVC 12 MB; ~49 min of history; 15.5k series

17.2 Operating

```
kubectll config use-context kind-workload-analyzer # make sure commands hit this cluster
kubectll -n monitoring port-forward svc/kps-grafana 3000:80
kubectll -n monitoring get secret grafana-admin -o jsonpath='{.data.admin-password}' | base64 -d; echo
kubectll -n monitoring port-forward svc/kps-kube-prometheus-stack-prometheus 9090
docker stop workload-analyzer-control-plane # pause the cluster (recovery UNVERIFIED)
docker start workload-analyzer-control-plane
```

Keep the laptop on AC power while collecting history: on battery it suspends, which creates gaps.

17.3 Rollback / removal

```
kubectl delete namespace learn
helm uninstall kps -n monitoring && kubectl delete namespace monitoring
kubectl get crd -o name | grep monitoring.coreos.com | xargs kubectl delete
kubectl delete -k deploy/metrics-server
kind delete cluster --name workload-analyzer          # removes everything, including Prometheus history
rm ~/.local/bin/{kubectl,kind,helm}
```

18. Open items and next phase

Item	Status
ADRs for accepted decisions (D-001, 002, 003, 005, 006, 007) with Phase 1 evidence	Due now ("after Phase 1")
D-004 (no application database)	Still PROPOSED
Cluster recovery after Docker restart / reboot	Phase 7 test
Grafana sidecars (~145 MiB, no limits)	Possible trim later
phase-1 branch	Fully merged; can be deleted
Phase 2 — demo workloads	Next: demo/ namespace with 8–9 deterministic workloads, each mapped to an expected finding in demo/expected-findings.yaml ; images pinned by digest

Phase 1 also showed which images to use in Phase 2: agnhost (stress , pause , netexec) covers CPU, memory, OOM and crash cases without Docker Hub, and resource-consumer gives shared, realistic load for the HPA workload.

Appendix A — Command cheat sheet

Task	Command
Everything in a namespace	<code>kubectl -n learn get deploy,rs,pods,svc --show-labels</code>
Why is a pod unhappy	<code>kubectl -n learn describe pod <p></code>
Recent events	<code>kubectl -n learn get events --sort-by=.lastTimestamp tail -20</code>
Last termination	<code>kubectl get pod <p> -o jsonpath='{.status.containerStatuses[0].lastState}'</code>
QoS class	<code>kubectl get pods -o custom-columns=NAME:.metadata.name,QoS:.status.qosClass</code>
Scheduling failures	<code>kubectl get events -A --field-selector reason=FailedScheduling</code>
Node reservations	<code>kubectl describe node workload-analyzer-control-plane sed -n '/Allocated resources/,/Events/p'</code>
Current usage	<code>kubectl top nodes; kubectl top pods -A --containers</code>
cgroup throttling	<code>kubectl exec <p> -- cat /sys/fs/cgroup/cpu.stat</code>
Rollout	<code>kubectl rollout status history undo deploy/<d></code>
Service endpoints	<code>kubectl get endpointslices -l kubernetes.io/service-name=<svc> -o wide</code>
HPA status	<code>kubectl get hpa <h> -o jsonpath='{.status}'; kubectl describe hpa <h></code>
RBAC check	<code>kubectl auth can-i <verb> <resource> --as=system:serviceaccount:<ns>:<sa></code>
Who am I	<code>kubectl auth whoami</code>
Node label / taint	<code>kubectl label node <n> k=v / k-; kubectl taint node <n> k=v:NoSchedule / ...-</code>
PromQL via API proxy	<code>kubectl get --raw "/api/v1/namespaces/monitoring/services/kps-kube-prometheus-stack-prometheus:9090/proxy/api/v1/query?query=up"</code>
CPU usage (PromQL)	<code>rate(container_cpu_usage_seconds_total{container!=""}[5m])</code>
Throttling ratio	<code>rate(container_cpu_cfs_throttled_periods_total{container!=""}[5m]) / rate(container_cpu_cfs_periods_total{container!=""}[5m])</code>
Memory peak	<code>max_over_time(container_memory_working_set_bytes{container!=""}[24h])</code>
OOM in window	<code>increase(kube_pod_container_status_restarts_total[1h]) > 0 and on(namespace,pod,container) kube_pod_container_status_last_terminated_reason{reason="OOMKilled"} == 1</code>
Crash loop in window	<code>max_over_time(kube_pod_container_status_waiting_reason{reason="CrashLoopBackOff"}[10m]) == 1</code>
Series count	<code>prometheus_tsdb_head_series</code>

Appendix B — Glossary

Term	Meaning
cAdvisor	Container-usage collector built into the kubelet; source of <code>container_*</code> metrics
cgroup v2	Linux kernel mechanism that enforces CPU/memory limits and counts usage and throttling
ClusterRole / Role	A set of permissions; ClusterRole is cluster-wide (needed for nodes), Role is per namespace
Context (kubectl)	Named cluster + user + namespace in kubeconfig; decides where commands go
CrashLoopBackOff	Container keeps exiting; the kubelet waits longer between restarts (up to 5 min)
EndpointSlice	List of ready Pod IPs behind a Service
Event	API object recording what happened to an object; expires after ~1 hour by default
HPA	HorizontalPodAutoscaler: changes the replica count from a metric; Utilization targets are relative to requests
kube-state-metrics (KSM)	Exports the state of API objects as Prometheus metrics
Liveness / readiness probe	Restart-if-failing check / receive-traffic-only-if-passing check
OOMKilled (exit 137)	Container killed by the kernel for exceeding its memory limit
PVC	PersistentVolumeClaim: request for storage that survives pod restarts
QoS class	BestEffort / Burstable / Guaranteed, from requests and limits; decides eviction order
Request / limit	Scheduler reservation / kernel-enforced ceiling
Series	One metric name + one unique label set in Prometheus; the main driver of Prometheus memory
ServiceMonitor	Operator object telling Prometheus what to scrape
Taint / toleration	Node-side "keep away" mark / pod-side permission to ignore it
Throttling	CPU paused for the rest of a 100 ms period after using its limit